Universidad Católica del Uruguay
Base de Datos 1

# INFORME DEL OBLIGATORIO

## Gestión del Torneo Interfacultades de Fútbol 5

Camila Insuasti - Santiago Pais
Profesor: Cristian Dinardi
Segundo Semestre 2026

# Indice

**Indice** 1
**Introducción** 2
**Modelo de Datos** 2
**Restricciones de integridad y decisión de no usar triggers** 4
Torneo y turnos 5
Equipos y planteles 6
Programación de partidos 7
Planilla, resultado y puntos 8
Sanciones 10
Usuarios y seguridad 10
Árbitros y espectadores 11
Decisiones ante casos que la letra no aclara 11
**Seguridad: usuarios y permisos** 11
Usuario administrador 12
Usuario planillero 12
Autenticación y gestión de credenciales 13
**Datos de prueba** 13
**Mejoras implementadas y consideradas** 14
**Validaciones en el backend** 14
Pruebas 15
**Reportes** 16
**Bitácora** 17
**Bibliografía** 19
**Anexo** 19

## Introducción

El siguiente informe está elaborado por estudiantes de primer año de la carrera Inteligencia Artificial y Ciencia de Datos de la Universidad Católica del Uruguay para Base de Datos 1 del segundo semestre del 2026.
El presente documento detalla el diseño e implementación de un sistema de información para gestionar el Torneo de Interfacultades de Fútbol 5 de la Universidad. El sistema cubre el calendario de partidos, el registro de planillas, los goles, las tarjetas y sanciones, y los reportes.
La base de datos utiliza lenguaje SQL y el *backend* está escrito en Python, sin ORM. Utiliza MySQL Connector para conectarse a la base de datos, python-dotenv para leer las credenciales desde un archivo .env y werkzeug para guardar las contraseñas con *hash*. A su vez, se utiliza Docker Compose, MySQL 8.0 y Python 3.14. La aplicación se utiliza por consola. El código está en un repositorio público de GitHub: https://github.com/santiagopais07-ui/torneo-interfacultades-bd
Para llevar a cabo la realización del mismo, el grupo optó por dividir tareas por participantes. Camila estuvo a cargo de la base de datos (modelo, script, datos de prueba y permisos), de las validaciones del *backend* y de los reportes, con sus pruebas automáticas. Santiago estuvo a cargo de la conexión, la creación de cuentas, los ABM y la parte de docker.

## Modelo de Datos

Partimos de las 14 tablas otorgadas en la letra y se agregaron tres más llamadas cambios, arbitro y espectador.
La tabla cambios registra quién corrigió un resultado y cuándo para dar trazabilidad.
La tabla arbitro se relaciona 1:N con partido, donde hay una columna ci_arbitro obligatoria. Hay un *UNIQUE* en dia, id_turno y ci_arbitro que impide que un árbitro dirija dos partidos al mismo tiempo. No se agregó una relación entre arbitro y tarjeta porque cada partido tiene un solo árbitro y cada tarjeta pertenece a un partido, el árbitro que la muestra se obtiene a través del partido y guardarlo en tarjeta duplicaría el dato.
La tabla espectador se relaciona 1:N con partido. Se simplificó de manera que si una persona asiste a dos partidos se registra dos veces y no se guarda su facultad, solo si es de la facultad o externa. Este tipo se valida con un *CHECK* en la base. La cédula del espectador es opcional porque un espectador externo puede no estar registrado.
Se consideró la idea de que los árbitros y espectadores pudieran recibir tarjetas pero se descartó ya que la letra establece que solo los jugadores de la planilla pueden recibirlas, y las sanciones y el *fair play* se calculan por jugador y por equipo.

Lo primero que se realizó fue un diagrama entidad-relación con todas las entidades, atributos y relaciones, luego se pasó el diagrama a tablas SQL en DataGrip.
A continuación se adjunta el diagrama:

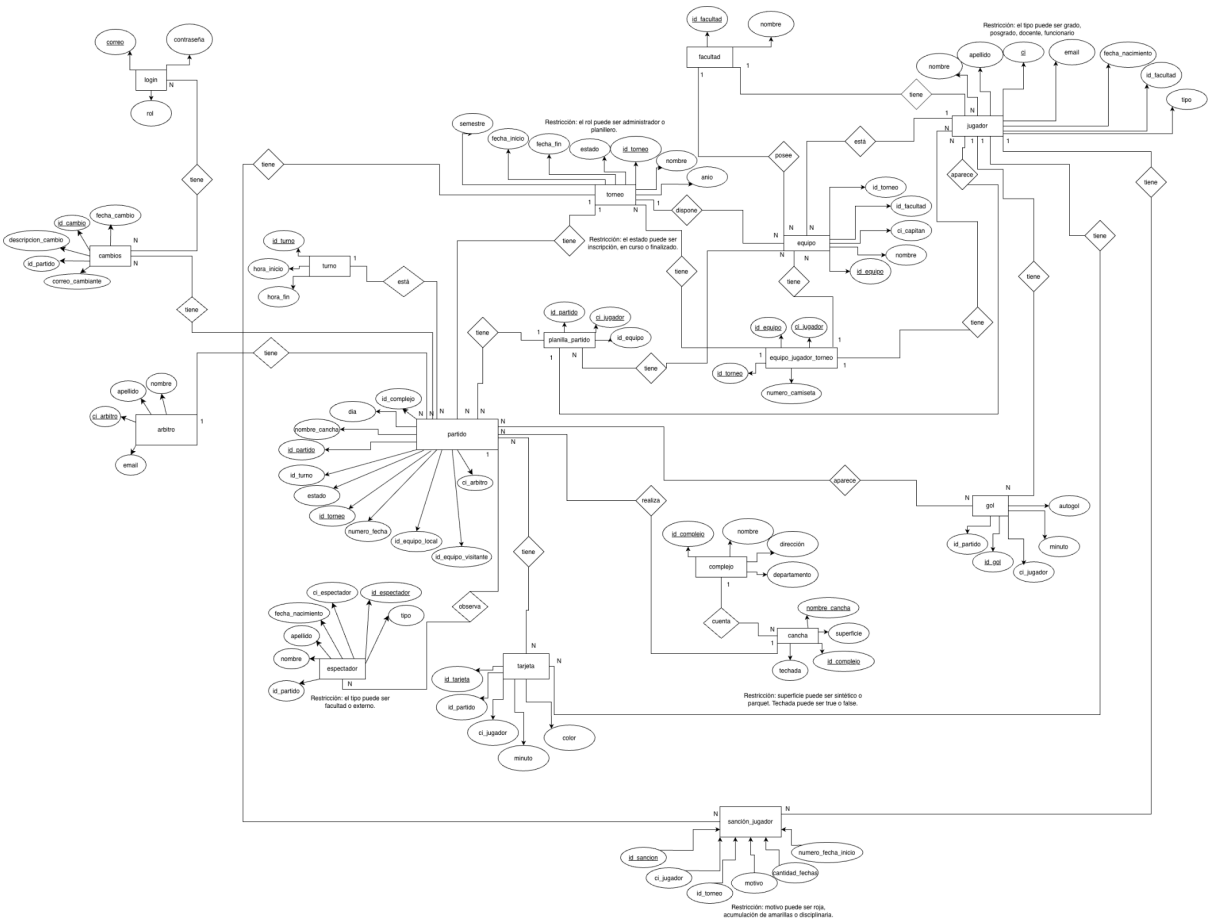



Figura 1: diagrama ER
Disponible en:
https://drive.google.com/file/d/1XlO4UNxK9AOFO6h2Lw5RdEFkIPPctMok/view?usp=sharing

A continuación se detallan las decisiones principales para las claves primarias:

- Se mantuvo la clave primaria de la letra en la tabla equipo_jugador_torneo (id_equipo, ci_jugador, id_torneo). Como por sí sola permitiría que un jugador esté en dos equipos de la misma edición, se agregó un *UNIQUE* en ci_jugador e id_torneo.
- A su vez, se agregó un *UNIQUE* a id_equipo y ci_jugador para que la tabla planilla_partido pueda referenciar el plantel con una clave foránea.
- En la tabla partido se mantuvo la clave primaria (id_partido, id_torneo) y se agregó un *UNIQUE* a id_partido, de esta manera, las tablas gol, tarjeta, planilla_partido y cambios pueden referenciar solo a id_partido sin cargar id_torneo en cada tabla.
- En la tabla cancha la clave primaria es (nombre_cancha, id_complejo) de modo que dos complejos pueden tener el mismo nombre de la cancha.

Se utilizaron claves foráneas compuestas para que la base de datos haga cumplir ciertas reglas sin necesidad de utilizar triggers:

- La tabla partido referencia a equipo (id_equipo, id_torneo) tanto para el local como para el visitante, así ambos equipos son de la misma edición que el partido.

- La tabla partido referencia a la tabla cancha (nombre_cancha, id_complejo).
- La tabla planilla_partido referencia a la tabla equipo_jugador_torneo (id_equipo, ci_jugador), de modo que solo figuran jugadores del plantel del equipo.
- Las tablas gol y tarjeta referencian a la tabla planilla_partido (id_partido, ci_jugador), y solo pueden convertir goles o recibir tarjetas quienes figuran en la planilla.

A su vez, la base tiene otras restricciones:

- La tabla torneo tiene un *UNIQUE* en anio y semestre para que haya una edición por semestre.
- La tabla partido tiene un *UNIQUE* en dia, id_turno, nombre_cancha e id_complejo para que una cancha no tenga dos partidos en el mismo día y turno.
- Hay *CHECK* para los valores de tipo, rol, estado, color de tarjeta, motivo de sanción y superficie, y para id_equipo_local <> id_equipo_visitante.
- Hay *NOT NULL* en las columnas que intervienen en claves foráneas compuestas y en *UNIQUE*, esto es ya que en MySQL esas restricciones no se controlan si alguna de sus columnas es nula.
- Un partido cancelado sigue ocupando la cancha, el día y el turno. Esto es por consecuencia del *UNIQUE*, lo cual es coherente con que tampoco se libera la fecha del equipo.
- Como *UNIQUE* no controla los nulos y en un torneo real todo partido tiene un árbitro, el árbitro es obligatorio en cada partido.

Otras decisiones de diseño:

- El resultado de un partido no se guarda, se calcula a partir de los goles para que no se desincronice. Los autogoles suman al rival.
- Los minutos de las tablas gol y tarjeta son enteros, no *time* ya que representan minutos de juego.
- Los cinco turnos son datos maestros y se cargaron en el script.

## Restricciones de integridad y decisión de no usar triggers

Se definieron treinta y dos reglas de integridad, agrupadas por categoría y para cada una se indica en qué capa se controla y por qué.
La aplicación se utiliza por consola por eso, lo que las tablas llaman interfaz se implementa en la consola como menús con opciones fijas, campos obligatorios, verificación del formato de los datos ingresados y mensajes de error claros. Sería una buena mejora a futuro agregar un *frontend*.

Varias de las siguientes restricciones se podrían resolver con triggers en la base de datos, pero como en el curso no lo vimos y como equipo no tenemos los conocimientos suficientes para realizarlo sin errores, decidimos optar por resolverlo de otras maneras. Las reglas controladas sólo en el *backend* no quedan protegidas

si alguien modifica la base de datos directamente, y se podrían pasar a triggers como mejora futura.

A continuación se especifica la regla, dónde se controla y el por qué. Está separado por categorías: Torneo y turnos, Equipos y planteles, Programación de partidos, Planilla, resultado y puntos, Sanciones, Usuarios y seguridad, Árbitros y espectadores, y se agrega una sección de Decisiones ante casos que la letra no aclara.

## Torneo y turnos

| | Regla | Dónde se controla | Por qué |
|---|---|---|---|
| 1 | Cada edición corresponde a un semestre, y no se repite | BD: *UNIQUE (anio, semestre)* y *CHECK* del semestre<br>*Backend*: valida<br>Interfaz: menú con opciones fijas | Es una combinación de columnas de la misma tabla |
| 2 | Las etapas son inscripción, en curso y finalizado, en ese orden | BD: *CHECK* de los valores<br>*Backend*: controla el avance y no deja volver atrás<br>Interfaz: opción de menú “pasar a la siguiente etapa”<br>Al pasar a “en curso” se exige que todos los equipos del torneo tengan al menos cinco jugadores | La BD solo conoce los valores posibles, no su orden |
| 3 | Los turnos duran una hora, entre las 18:00 y las 23:00 | BD: *CHECK* sobre las horas y datos maestros con los 5 turnos<br>*Backend*: valida que el turno exista<br>Interfaz: menú con opciones fijas | Las dos horas están en la misma fila |
| 4 | Los partidos se agrupan en fechas numeradas | BD: *NOT NULL* y *CHECK (numero_fecha >= 1)*<br>*Backend*: valida<br>Interfaz: valida | Es una condición sobre una columna |

## Equipos y planteles

| | Regla | Dónde | Por qué |
|---|---|---|---|
| 5 | Cada equipo tiene nombre y representa a una facultad | BD: *NOT NULL* y *FOREIGN KEY* a facultad<br>*Backend*: campos obligatorios<br>Interfaz: campos obligatorios | Es integridad básica de la tabla |
| 6 | El tipo de jugador es grado, posgrado, docente o funcionario | BD: *CHECK*<br>*Backend*: valida<br>Interfaz: menú con opciones fijas | Valor fijo de una sola columna |
| 7 | El capitán de un equipo debe ser uno de sus jugadores | BD: *FOREIGN KEY* a jugador, que solo garantiza que existe<br>*Backend*: verifica que esté en el plantel y no deja sacarlo mientras sea capitán<br>Interfaz: menú con opciones fijas | Estar en el plantel se mira en otra tabla. Se cargan equipo, capitán y plantel en la misma transacción. |
| 8 | Cada plantel admite como máximo 2 docentes/funcionarios | *Backend*: cuenta antes de agregar<br>Interfaz: muestra el mensaje de error de la validación | Hay que contar filas del plantel y mirar en la tabla jugador el tipo |
| 9 | El plantel tiene entre 5 y 10 jugadores | *Backend*: corta el jugador 11 al agregar, y exige 5 o más al pasar el torneo a “en curso” y al programar un partido<br>Interfaz: muestra el mensaje de error al intentar agregar el jugador 11 | Los jugadores se cargan de a uno, así que el mínimo no se puede exigir en cada alta |
| 10 | Los jugadores pertenecen a la facultad del equipo | *Backend*: compara el id_facultad de la tabla equipo con el id_facultad de la tabla jugador<br>Interfaz: menú solo con jugadores de esa | Compara dos tablas |

| | | | |
|---|---|---|---|
| | | facultad | |
| 11 | Un jugador no integra más de un equipo en la misma edición | BD: *UNIQUE (ci_jugador, id_torneo)*<br>*Backend*: muestra un mensaje claro si falla | Es una restricción de dos columnas de la misma tabla |
| 12 | Los planteles sólo se modifican en etapa de inscripción | *Backend*: consulta en la tabla torneo el estado en altas, bajas y cambios de camiseta | Depende del estado del torneo, que está en otra tabla |

## Programación de partidos

| | Regla | Dónde se controla | Por qué |
|---|---|---|---|
| 13 | Ambos equipos son de la misma edición que el partido | BD: *UNIQUE (id_equipo, id_torneo)* en equipo y *FOREIGN KEY* compuestas desde partido hacia (equipo, torneo)<br>*Backend*: valida<br>Interfaz: sólo ofrece equipos de la edición | Una FK compuesta lo garantiza sin código |
| 14 | Ambos equipos tienen al menos 5 jugadores | *Backend*: cuenta el plantel de cada uno<br>Interfaz: muestra el mensaje de error de la validación | Hay que contar filas de otra tabla |
| 15 | Un equipo no se enfrenta a sí mismo | BD: *CHECK (id_equipo_local <> id_equipo_visitante)*<br>*Backend*: valida<br>Interfaz: valida | Compara dos columnas de la misma fila |
| 16 | Un equipo no repite rival en la edición, salvo que el partido anterior esté cancelado | *Backend*: busca partidos previos entre los dos, en cualquiera de los dos órdenes, ignorando los cancelados<br>Interfaz: muestra el mensaje de error de la validación<br>Los partidos postergados sí cuentan como partido ya existente. Solo se | Hay que mirar otras filas, y un *UNIQUE* no distingue local de visitante ni el estado |

| | | | |
|---|---|---|---|
| | | ignoran los cancelados. | |
| 17 | Un equipo no juega más de un partido por fecha | *Backend*: busca partidos del equipo, como local o visitante, en esa fecha<br>Interfaz: muestra el mensaje de error de la validación<br>Cuentan los partidos en cualquier estado (programado, jugado, postergado o cancelado): un partido cancelado o postergado no libera la fecha del equipo. | Compara con otras filas |
| 18 | Una cancha no tiene más de un partido el mismo día y turno | BD: *UNIQUE(dia, id_turno, nombre_cancha, id_complejo)*<br>*Backend*: valida<br>Interfaz: el menú muestra canchas libres<br>Un partido cancelado sigue ocupando su cancha, día y turno (ver decisiones). | Es única una combinación de columnas de la misma tabla |

## Planilla, resultado y puntos

| | Regla | Dónde se controla | Por qué |
|---|---|---|---|
| 19 | Solo figuran jugadores del plantel de alguno de los dos equipos, y el equipo de la planilla es uno de los dos del partido | BD: *FOREIGN KEY* compuesta de la tabla planilla_partido hacia la tabla equipo_jugador_torneo<br>*Backend*: valida el equipo<br>Interfaz: menú de solo esos jugadores | La *FOREIGN KEY* compuesta impide jugadores ajenos al plantel |
| 20 | Un jugador suspendido no figura en la planilla de las fechas que abarca su sanción | *Backend*: busca en la tabla sancion_jugador si el número de fecha del partido cae dentro de la sanción<br>Interfaz: muestra el mensaje de error de la | Cruza sanciones con el número de fecha del partido |

| | | | |
|---|---|---|---|
| | | validación | |
| 21 | Solo convierten goles o reciben tarjetas jugadores que figuran en la planilla | BD: *FOREIGN KEY* compuesta (id_partido, ci_jugador) desde la tabla gol y la tabla tarjeta hacia la tabla planilla_partido<br>*Backend*: valida<br>Interfaz: menú solo de la planilla | La *FOREIGN KEY* compuesta lo resuelve en la BD |
| 22 | El resultado se calcula con los goles registrados (un autogol suma al rival) | *Backend*: consulta SQL, sin guardar el resultado | Si se guardara, podría desincronizarse de los goles |
| 23 | Un partido solo se marca como jugado si su día y turno ya pasaron | *Backend*: compara con la hora actual y hora_fin en la tabla turno<br>Interfaz: no ofrece la opción antes de tiempo<br>Las planillas, goles y tarjetas solo se registran en partidos ya disputados que no estén cancelados ni postergados | *CHECK* no admite la hora actual |
| 24 | Un partido jugado no se puede eliminar | BD: sin permiso *DELETE* sobre partido para los usuarios de la aplicación<br>*Backend*: no ofrece borrar jugados | Es una restricción sobre una operación, no sobre un dato.<br>Nadie puede borrar partidos, la baja de un partido es pasarlo a estado cancelado |
| 25 | El resultado solo se corrige dentro de las cuarenta y ocho horas posteriores al partido | *Backend*: calcula día + hora_fin + cuarenta y ocho horas y registra cada corrección en cambios<br>Si la corrección modifica una tarjeta, la sanción asociada se recalcula en la misma transacción (ver regla 27). | Depende de la hora actual y deja trazabilidad |
| 26 | Victoria 3 puntos, empate 1, derrota 0. | Consulta del reporte | Es un cálculo, no una restricción |

| | | | |
|---|---|---|---|
| | Orden por puntos, luego diferencia de goles y luego goles a favor | | |

## Sanciones

| | Regla | Dónde se controla | Por qué |
|---|---|---|---|
| 27 | Una roja, o 3 amarillas en la edición, sancionan automáticamente con 1 fecha, que se cumple en la fecha siguiente | *Backend*: al guardar una tarjeta, cuenta las de esa edición y crea la sanción con numero_fecha_inicio = fecha del partido + 1, en la misma transacción.<br>Si una tarjeta se corrige o elimina dentro de las cuarenta y ocho horas, el *backend* recalcula la sanción en la misma transacción y lo registra en cambios | Tiene que dispararse sola y hay que contar filas de otra tabla |
| 28 | La organización puede aplicar sanciones disciplinarias de varias fechas | BD: *CHECK (cantidad_fechas >= 1)*<br>*Backend*: solo administradores<br>Interfaz: pantalla de carga con campos obligatorios | La cantidad es una columna, y quién la aplica es una regla de permisos |

## Usuarios y seguridad

| | Regla | Dónde se controla | Por qué |
|---|---|---|---|
| 29 | Dos tipos de usuario: el administrador gestiona todo, y el planillero solo registra planillas, goles y tarjetas | BD: dos usuarios de MySQL con *GRANT* distintos<br>*Backend*: verifica el rol en cada menú<br>El administrador hace el ABM de árbitros y espectadores, el planillero solo los lee<br>Interfaz: en cada opción | La letra pide seguridad a nivel de BD, y las otras capas la refuerzan |

| | | | |
|---|---|---|---|
| | | del menú | |
| 30 | Los campos se validan en las tres capas | BD, *Backend* e Interfaz: *NOT NULL, CHECK, UNIQUE, FK* | Cada capa se protege aunque falle la anterior |

## Árbitros y espectadores

| | Regla | Dónde se controla | Por qué |
|---|---|---|---|
| 31 | Cada partido tiene un árbitro, y un árbitro no dirige dos partidos el mismo día y turno | BD: ci_arbitro *NOT NULL* y *FOREIGN KEY* a arbitro en partido, y *UNIQUE (dia, id_turno, ci_arbitro)*<br>*Backend*: captura los errores 1062 y 1452 y muestra un mensaje claro, el árbitro se elige de la lista de árbitros existentes | Es una combinación de columnas de la misma tabla, igual que la cancha. La columna es obligatoria porque en MySQL un *UNIQUE* no controla los valores nulos |
| 32 | El tipo de espectador es facultad o externo | BD: *CHECK*<br>*Backend*: valida<br>Interfaz: opciones fijas en el menú | Es un valor fijo de una sola columna |

## Decisiones ante casos que la letra no aclara

1. Sanciones por acumulación de amarillas: cada vez que el total de amarillas del jugador en la edición llega a un múltiplo de tres se genera una sanción de una fecha
2. Partidos cancelados o postergados: no liberan la fecha del equipo. El equipo no puede jugar otro partido en esa fecha ni descansa, y un partido cancelado sigue ocupando su cancha, día y turno
3. Corrección de tarjetas: si se corrige una tarjeta dentro de las cuarenta y ocho horas, la sanción asociada se corrige automáticamente, en la misma transacción, y queda registrada en la tabla cambios.
4. Baja de partidos: nadie puede borrar partidos. La baja es pasarlos a estado “cancelado”.

## Seguridad: usuarios y permisos

Se crearon dos usuarios de MySQL con permisos diferenciados: admin_torneo y planillero_torneo. La separación de permisos se realizó de manera que cada usuario tenga únicamente los permisos necesarios para cumplir sus funciones. Esto permite reducir el riesgo de modificaciones o eliminaciones accidentales y proteger la integridad de los datos.

A continuación se especifica qué permisos tiene cada uno y por qué.

## Usuario administrador

El administrador del torneo cuenta con permisos de lectura, inserción, modificación y eliminación (SELECT, INSERT, UPDATE y DELETE) sobre las tablas de gestión. Esto se debe a que es el responsable de administrar la información general del torneo y realizar las operaciones de mantenimiento necesarias.
Aunque es administrador, no tiene todos los permisos. A continuación se especifican algunas restricciones específicas:

- Sobre la tabla partido, el administrador no tiene permiso de eliminación ya que un partido que ya fue disputado no debe eliminarse de la base de datos. En caso de que un partido no pueda realizarse, su baja se gestiona mediante un cambio de estado a “cancelado”. De esta forma, se conserva la información del partido y su historial.
- Sobre la tabla cambios, el administrador únicamente puede leer e insertar registros. Esta tabla funciona como una bitácora de trazabilidad, por lo que los registros no deben modificarse ni eliminarse. Esto permite mantener un historial confiable de las operaciones realizadas.
- Sobre la tabla login, el administrador puede leer, insertar y modificar registros, pero no eliminarlos. Estos permisos se justifican porque es el responsable de gestionar las cuentas de los usuarios y sus contraseñas, mientras que se busca conservar la información de las cuentas registradas.

A su vez, el administrador es el único usuario que puede realizar el alta, baja y modificación de árbitros y espectadores. El planillero únicamente puede consultar sus datos, ya que no es responsable de su gestión.

## Usuario planillero

El planillero cuenta con permisos de lectura, inserción, modificación y eliminación sobre las tablas planilla_partido, gol y tarjeta. Estos permisos le permiten registrar y corregir la información de los partidos dentro del plazo establecido de cuarenta y ocho horas, sin otorgarle acceso para modificar otros datos de gestión del torneo.
Sobre la tabla partido, el planillero únicamente puede leer los registros y modificar la columna estado. Esto le permite marcar un partido como jugado una vez finalizado, sin modificar otros datos del encuentro, como su fecha, equipos o información general.
En la tabla cambios, dispone de permisos de lectura e inserción. De esta manera, puede consultar el historial de modificaciones y registrar nuevas operaciones, pero no alterar ni eliminar los registros existentes, manteniendo la trazabilidad de las acciones realizadas.
En el resto de las tablas, el planillero cuenta únicamente con permisos de lectura. La excepción es la tabla sancion_jugador sobre la cual dispone de permisos de escritura, debido a que el *backend* crea y corrige automáticamente las sanciones

cuando el planillero guarda una tarjeta. Sin embargo, la base de datos no distingue entre una sanción automática, generada a partir de una tarjeta y una sanción disciplinaria aplicada manualmente. Por este motivo, la restricción que establece que únicamente los administradores pueden aplicar sanciones disciplinarias se controla desde el *backend*, donde se valida el rol del usuario antes de realizar la operación.
El planillero no tiene acceso a la tabla login, ya que no necesita consultar ni gestionar las credenciales de los usuarios para cumplir con sus funciones.

## Autenticación y gestión de credenciales

Como únicamente el administrador tiene permisos de lectura sobre la tabla login, el proceso de autenticación se realiza utilizando inicialmente la conexión correspondiente al administrador. El *backend* verifica el correo electrónico y la contraseña ingresados y una vez validada la identidad del usuario, establece la conexión correspondiente a su rol.
Las contraseñas almacenadas en la tabla login se guardan mediante un proceso de *hashing*, evitando que queden almacenadas en texto plano. Esto permite proteger las credenciales en caso de que alguien acceda directamente a la información de la base de datos.
Las cuentas de MySQL como admin_torneo y planillero_torneo, y sus permisos se crean en el script SQL. Las cuentas de la aplicación, admin@correo.ucu.edu.uy y planillero@correo.ucu.edu.uy se crean automáticamente la primera vez que se inicia el programa, se piden las contraseñas por consola y se guarda solo su *hash*.
Las credenciales utilizadas para establecer las conexiones se almacenan en un archivo .env, que no se incluye en el repositorio. De esta forma, se evita exponer información sensible al compartir el código del proyecto. El archivo .env se usa para correr sin Docker, con Docker las credenciales vienen del archivo llamado compose.yaml.
Por último, el script SQL contiene las cláusulas *IDENTIFIED BY 'usuario1'* e *IDENTIFIED BY 'usuario2'* que corresponden a claves de demostración, también utilizadas en el entorno Docker local y no deben usarse en un despliegue real.

## Datos de prueba

El script incluye datos suficientes para ejecutar todos los reportes, teniendo seis facultades, seis equipos con siete jugadores cada uno (el séptimo es un suplente que nunca figura en una planilla), un torneo en curso, tres complejos con cinco canchas, tres árbitros y seis espectadores (tres de la facultad y tres externos). De los partidos, nueve están jugados (tres fechas completas) y hay uno programado, uno postergado y uno cancelado. Además, se cargaron goles (incluido un autogol), tarjetas y cuatro sanciones (dos rojas, una por acumulación de amarillas y una disciplinaria). Todos los datos ingresados son ficticios.

Las planillas excluyen a los suplentes y a los jugadores suspendidos en cada fecha. Con estos datos, la tabla de posiciones tiene un empate en puntos que se desempata por diferencia de goles.

## Mejoras implementadas y consideradas

Las mejoras implementadas son las siguientes:

- La tabla cambios registra quién corrigió un resultado y cuándo, para dar la trazabilidad.
- Se implementan claves foráneas compuestas
- Varios *UNIQUE*
- Permisos por columna
- Las tablas árbitro y espectador
- Pruebas automáticas
- Los doce reportes y sus pruebas automáticas.

Las mejoras consideradas a futuro para mejorar el sistema:

- Implementar con triggers las reglas que hoy valida el backend.
- Una tabla de fechas o jornadas del torneo, para no repetir numero_fecha en las tablas partido y sancion_jugador.
- Vistas SQL para los reportes.
- Más de un árbitro por partido.
- Que un espectador asista a varios partidos, creando una tabla intermedia.
- Guardar en la tabla cambios toda la auditoría y no solo las correcciones de resultados.

## Validaciones en el *backend*

Las reglas que no pueden resolverse con restricciones declarativas se implementaron en un archivo llamado reglas.py. Cada una es una función que recibe el cursor, consulta la base y lanza un *ValueError* con un mensaje claro si la regla no se cumple. Se llaman antes de cada *INSERT*, *UPDATE* o *DELETE*, dentro de la misma transacción y con un único *commit* al final.

A continuación se especifican las decisiones tomadas:

- El alta de planteles bloquea la fila del equipo con *SELECT… FOR UPDATE*, para que dos altas simultáneas no superen el máximo de diez jugadores.
- Las funciones que dependen de la hora reciben un parámetro *ahora*, para probarlas con fechas inventadas.
- Las sanciones automáticas se recalculan desde cero a partir de todas las tarjetas del jugador en la edición, sin tocar las disciplinarias, de modo que el alta, la corrección y la baja de una tarjeta usan la misma función.

## Pruebas

Se realizaron pruebas automáticas, una por área del sistema, que se ejecutan dentro del contenedor de la aplicación. A menos que se indique lo contrario, trabajan dentro de una transacción que se revierte al final, por lo que no se modifican los datos. Los datos de la revisión del cinco de noviembre están en el archivo PRUEBAS.md.

- probar_reglas.py ejecuta sesenta pruebas sobre las funciones de reglas.py: máximo y mínimo del plantel, docentes y funcionarios, facultad del jugador, capitán, jugadores suspendidos, ventana de cuarenta y ocho horas y recálculo de sanciones. Usa fechas inventadas mediante el parámetro *ahora*.
- probar_reportes.py ejecuta doce pruebas, una por reporte, que comparan la salida con el resultado esperado calculado a mano sobre los datos de prueba. Solo lee datos.
- probar_partidos.py ejecuta treinta pruebas sobre la programación y reprogramación de partidos: conflictos de cancha y de árbitro, rivales y fechas, y protección de los partidos jugados o con planilla.
- probar_planillas.py ejecuta cuarenta pruebas sobre participantes, goles, autogoles, tarjetas, suspensiones automáticas y disciplinarias, correcciones y límite de cuarenta y ocho horas. Incluye operaciones con la cuenta MySQL real del planillero.
- probar_torneos.py ejecuta dieciséis pruebas sobre las ediciones, el cambio de etapas y los límites del plantel.
- probar_usuarios.py ejecuta siete pruebas de autenticación: contraseña correcta con *hash*, contraseña incorrecta, un intento de inyección SQL, creación de las cuentas que faltan y conservación de las existentes.
- probar_integracion.py recorre doce grupos de operaciones completas con datos temporales, combinando varios módulos y revierte todo.
- probar_menu_completo.py navega los nueve submenús administrativos y los doce reportes desde la consola, comprueba que MySQL rechaza ocho operaciones administrativas al planillero y realiza un alta y una baja reales de un jugador temporal. Verifica desde otra conexión que se guardaron y que todas las filas originales se conservan.

Además se reinició la base y se comprobó que los datos se mantuvieron. Los saltos en los identificadores automáticos después de probar son normales. Las pruebas no cubren concurrencia de múltiples usuarios ni fallos de red o *hardware*.

A continuación se muestra una tabla con la regla y con qué función se valida:

| Regla | Función |
|---|---|
| 2 | validar_cambio_etapa |
| 7 | validar_capitan_en_plantel y validar_baja_jugador_plantel |

| | |
|---|---|
| 8 | validar_max_docentes_funcionarios |
| 9 | validar_max_plantel el máximo y validar_min_jugadores el mínimo |
| 10 | validar_misma_facultad |
| 12 | validar_torneo_en_inscripcion |
| 14 | validar_min_jugadores |
| 16 | validar_rival_no_repetido |
| 17 | validar_un_partido_por_fecha |
| 19 | validar_equipo_del_partido y la clave foránea compuesta |
| 20 | validar_jugador_no_suspendido |
| 23 | validar_partido_disputado y validar_partido_registrable |
| 24 | validar_baja_partido |
| 25 | validar_ventana_correccion, validar_partido_registrable y registrar_cambio |
| 27 | recalcular_sanciones_automaticas |
| 28 | validar_sancion_disciplinaria |

Las siguientes funciones agrupan a otras para que la aplicación llame a una sola por operación:

| Función | Reúne reglas |
|---|---|
| validar_alta_plantel | 12, 10, el máximo de la 9 y 8 |
| validar_alta_partido | 14, 16 y 17 |
| validar_alta_planilla | 23 y 25 vía validar_partido_registrable, 19 y 20 |
| validar_partido_registrable | 23, 25 y que el partido no esté cancelado ni postergado |

## Reportes

El sistema ofrece doce reportes, nueve de la letra y tres consultas adicionales. Cada reporte es una opción del menú de la consola, el programa ejecuta una consulta SQL y muestra el resultado. Las consultas están en el archivo reportes.py y se ejecutan con el usuario administrador, que tiene permiso de lectura sobre todas las

tablas. El planillero no accede a los reportes porque su rol está limitado. Los reportes solo leen datos.
A continuación se especifican unas decisiones comunes a todos los reportes:

- Se calculan para una edición del torneo elegida por el usuario. En el reporte de jugadores por facultad y tipo se cuentan los jugadores inscriptos en algún plantel de esa edición.
- Para goles, posiciones, promedios, canchas y turnos solo se cuentan partidos en estado jugado.
- Un gol suma al equipo con el que el jugador figura en la planilla del partido. Un autogol suma al rival y no se cuenta como gol del jugador.
- La tabla de posiciones se ordena por puntos, diferencia de goles y goles a favor. Los equipos que siguen igualados comparten posición.
- En el *fair play*, los equipos con el mismo puntaje comparten posición.
- En los goleadores, ante igual cantidad de goles queda mejor el jugador con menos puntos de *fair play* (uno por amarilla, tres por roja). Si el empate persiste, se ordena por apellido solo para que el resultado sea siempre el mismo.

Los reportes disponibles son:

1. Tabla de posiciones con partidos jugados, ganados, empatados y perdidos, goles a favor y en contra, diferencia y puntos.
2. Tabla de goleadores, los diez con más goles.
3. *Fair play* por equipo, puntos por amarillas y rojas.
4. Promedio de goles por partido en cada fecha y en toda la edición.
5. Canchas y turnos más utilizados.
6. Cantidad de jugadores por facultad y tipo.
7. Jugadores inscriptos que nunca figuraron en una planilla.
8. Cantidad de sanciones por equipo y motivo.
9. Porcentaje de partidos jugados, programados, postergados y cancelados.
10. Partidos por árbitro y estado.
11. Tarjetas mostradas por el árbitro.
12. Espectadores por partido y tipo.

Los reportes 10, 11 y 12 dan uso de las tablas árbitro y espectador. El reporte once se obtiene uniendo tarjeta, partido y árbitro sin guardar el árbitro en tarjeta.
Cada reporte se comprueba con pruebas automáticas, ver sección Pruebas.

## Bitácora

El martes 20/10 se leyó la letra y se dividió las tareas dependiendo la afinidad y conocimiento de cada uno entre los participantes del grupo.
El domingo 25/10 Camila armó el diagrama entidad-relación y las tablas en DataGrip. Se revisaron errores de claves primarias duplicadas y claves foráneas faltantes y se definió la tabla cambios. Se armaron las tablas de reglas de negocio y se decidió no usar triggers. Se definió qué reglas pasan al backend y cuáles se resuelven con claves foráneas compuestas. Se escribió el documento de

restricciones de integridad y se definieron las decisiones sobre los casos que la letra no aclara.
El lunes 26/10 se cargaron los datos maestros y los datos de prueba, se verificaron los conteos de cada tabla. Se crearon los usuarios admin_torneo y planillero_torneo con sus respectivos permisos.
El mismo lunes Santiago creó el repositorio público torneo-interfacultades-bd en GitHub y clonó en PyCharm. Configuró el entorno virtual de Python e instaló Flask, MySQL Connector y python-dotenv. Revisó los archivos SQL y las restricciones de integridad. Importó la base de datos y corrigió errores que impedían la creación de las tablas login y cambios. Verificó la creación de las 15 tablas y de los datos de prueba. Creó y probó la conexión.py para establecer conexiones con MySQL según el rol de usuario.
El martes 27/10 Santiago configuró el archivo .env con los datos de conexión. Además configuró el archivo .gitignore para evitar subir información sensible al repositorio. Luego creó y ejecutó crear_usuarios.py para registrar las cuentas de administrador y planillero. Arregló el almacenamiento de las contraseñas mediante hashes en la tabla login. Verificó las conexiones y los permisos de ambos usuarios.
El miércoles 28/10 Santiago hizo una prueba inicial con Flask.
El jueves 29/10 Camila armó el borrador del informe.
El viernes 30/10 se presentó el avance a los profesores, donde se descartó la implementación de Flask al decidir usar la consola.
Luego, el mismo día Camila agregó la tabla árbitro y espectador, lo que obligó a modificar claves foráneas y más.
El sábado 31/10 Camila implementó las validaciones del *backend*, reglas.py y sus pruebas automáticas, probar_reglas.py sesenta pruebas y todas correctas.
El domingo 01/11 se modificó el informe y el diagrama ER.
El martes 3/11 Camila definió los doce reportes con resultados esperados y los criterios de desempate.
El miércoles 4 y jueves 5/11 Santiago se encargó del ABM por consola, la parte del Docker, probar_integracion, probar_menu_completo y la revisión de PRUEBAS.md.
El jueves 5/11 Camila terminó de editar el informe con las últimas modificaciones y generó el instructivo.

## Bibliografía

GeeksforGeeks. (2025/07/19). Connect MySQL database using MySQL-Connector Python. https://www-geeksforgeeks-org.translate.goog/python/connect-mysql-database-using-mysql-connector-python
TuxCare. (2026/10/02). Werkzeug Security: CVE, riesgos y buenas prácticas en 2026. https://tuxcare.com/es/blog/werkzeug-security/
GeeksforGeeks. (2025/07/23). Using Python Environment Variables with Python Dotenv. https://www-geeksforgeeks-org.translate.goog/python/using-python-environment-variables-with-python-dotenv
Material del curso Base de Datos 1, UCU, 2026.
Repositorio GitHub: https://github.com/santiagopais07-ui/torneo-interfacultades-bd
Se utilizaron asistentes de IA como apoyo para escribir y revisar código, consultas y el informe. Las decisiones de diseño fueron tomadas por el grupo y el código fue revisado y probado por sus integrantes.

## Anexo

Instructivo completo de ejecución. Anexo A